

**UD 00**

# **Actividades evaluables 01**

**Ángel Esteban Parra**

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| Actividad 01 .....                                                                   | 3  |
| Clonar una lista (ShallowCopy) .....                                                 | 3  |
| Clonar una lista (DeepCopy) .....                                                    | 3  |
| Diferencia DeepCopy y ShallowCopy .....                                              | 4  |
| Añadir un elemento a la lista .....                                                  | 4  |
| Quitar un elemento de la lista .....                                                 | 5  |
| Quitar con .remove .....                                                             | 5  |
| Quitar con .pop.....                                                                 | 5  |
| Quitar con del.....                                                                  | 6  |
| Crear una nueva lista con los últimos 4 elementos de una lista .....                 | 6  |
| Convertir las palabras de una cadena separada por espacios a una lista .....         | 6  |
| Comentarios con una línea .....                                                      | 7  |
| Comentarios multilínea .....                                                         | 7  |
| Actividad 02 .....                                                                   | 7  |
| Función que reciba dos números y devuelva la suma .....                              | 8  |
| Función que recibe una lista y la modifique doblando sus valores sin devolver nada . | 8  |
| Función que recibe una lista, la copia y la dobla sin modificar la original .....    | 9  |
| Actividad 03.....                                                                    | 9  |
| Almacenar un usuario con su contraseña pasándolo a Hash .....                        | 10 |
| Usando una lista .....                                                               | 10 |
| Usando un diccionario .....                                                          | 11 |
| Actividad 04.....                                                                    | 12 |
| Operador “is” .....                                                                  | 12 |
| Operador “not” .....                                                                 | 12 |
| Operador “in” .....                                                                  | 13 |
| Actividad 05.....                                                                    | 14 |
| Introducir parámetros .....                                                          | 14 |
| Sobrecarga de funciones .....                                                        | 15 |
| Actividad 06.....                                                                    | 17 |
| Actividad 07.....                                                                    | 18 |
| Enlace GitHub.....                                                                   | 20 |



## UD 00

### Actividades evaluables 01

Ángel Esteban Parra

## Actividad 01

### Clonar una lista (ShallowCopy).

```
import copy
import hashlib

lista1 = [1, 2, ["manzana", "pera"]]
lista2 = copy.copy(lista1) # Creamos las listas y hacemos ShallowCopy.

print("Listas: ")
print(lista1)
print(lista2) # Comprobamos que son iguales.

lista1[2].append("melon")
"""
Queremos añadir el melon a la lista de frutas, por lo que
ponemos el [2] que es la posición donde está y usamos .append que añade a la siguiente
posición disponible dentro de la caja.
"""

print("Listas con cambios: ")
print("Lista 1 original: ")
print(lista1)
print("Lista 2 copiada: ")
print(lista2)
```

Resultado:

```
Listas:
[1, 2, ['manzana', 'pera']]
[1, 2, ['manzana', 'pera']]
Listas con cambios:
Lista 1 original:
[1, 2, ['manzana', 'pera', 'melon']]
Lista 2 copiada:
[1, 2, ['manzana', 'pera', 'melon']]
```

## Clonar una lista (DeepCopy)

```
lista3 = copy.deepcopy(lista1)
lista1[2].append("naranja")

print ("Listas con más cambios: ")
print("Lista 1 original: ")
print(lista1)
print("Lista 3 copiada: ")
print(lista3)

"""
Como se puede ver aquí, hemos hecho una DeepCopy de la lista 1 y es totalmente
independiente, es decir que por muchos cambios que le haga ahora a la lista 1 no se
replicarán en la lista 3
"""
```

Resultado:

```
Listas con más cambios:
Lista 1 original:
[1, 2, ['manzana', 'pera', 'melon', 'naranja']]
Lista 3 copiada:
[1, 2, ['manzana', 'pera', 'melon']]
```

## Diferencia DeepCopy y ShallowCopy

A diferencia de ShallowCopy, DeepCopy solo copia la lista y cuando haces un cambio en la original, no se altera la copia. Es totalmente independiente.

## Añadir un elemento a la lista

```
lista1 = [1, 2, ["manzana", "pera"]]
```

```
lista1[2].append("melon")
"""
Queremos añadir el melon a la lista de frutas, por lo que
ponemos el [2] que es la posición donde está y usamos .append que añade a la siguiente
posición disponible dentro de la caja.
"""
```

Resultado:

```
Listas con cambios:  
Lista 1 original:  
[1, 2, ['manzana', 'pera', 'melon']]
```

## Quitar un elemento de la lista

### Quitar con .remove

```
lista4 = ["lechuga","zanahoria", "remolacha", "lechuga"]  
print("Lista 4: ")  
print(lista4)  
  
lista4.remove("lechuga")  
print("Ahora borramos una de las lechugas: ")  
print(lista4)  
  
# Para borrar elementos en la lista usaremos .remove y eliminará la primera  
# aparición del elemento que elijamos borrar.
```

Resultado:

```
Lista 4:  
['lechuga', 'zanahoria', 'remolacha', 'lechuga']  
Ahora borramos una de las lechugas:  
['zanahoria', 'remolacha', 'lechuga']
```

### Quitar con .pop

```
print("Ahora vamos a borrar un elemento y guardarlo: ")  
print(lista4)  
eliminadoL4 = lista4.pop(1)  
print("Eliminado: ")  
print(eliminadoL4)  
print("Lista actual: ")  
print(lista4)  
  
# El comando .pop borra un elemento de la lista pero lo guarda en otra variable para  
# el uso que le quieras dar.
```

Resultado:

```
Ahora vamos a borrar un elemento y guardarlo:  
['zanahoria', 'remolacha', 'lechuga']  
Eliminado:  
remolacha  
Lista actual:  
['zanahoria', 'lechuga']
```

## Quitar con del

```
print ("Para finalizar vamos a borrar un último elemento eligiendo su posición: ")  
print(lista4)  
del lista4[0]  
print ("Lista con elemento borrado: ")  
print(lista4)  
  
# Este comando te permite borrar algo eligiendo la posición.
```

Resultado:

```
Para finalizar vamos a borrar un último elemento eligiendo su posición:  
['zanahoria', 'lechuga']  
Lista con elemento borrado:  
['lechuga']
```

## Crear una nueva lista con los últimos 4 elementos de una lista

```
lista5 = [13, 1, 6, 7, 12, 4, 8, 9]  
lista6 = lista5[-4:]  
  
print("Nueva lista: ")  
print(lista5)  
print("Lista con los últimos 4 números: ")  
print(lista6) # ShallowCopy  
  
# Al poner lista6 = lista5[-4:] usa la técnica de 'slicing'. Esta técnica, cuando  
# le pones el índice negativo le estás pidiendo que cuente desde atrás y con los dos  
# puntos que pille los 4.
```

Resultado:

```
Nueva lista:  
[13, 1, 6, 7, 12, 4, 8, 9]  
Lista con los últimos 4 números:  
[12, 4, 8, 9]
```

## Convertir las palabras de una cadena separada por espacios a una lista

```
cadena = "me gustan las albondigas"
listaCadena = cadena.split()

print("Oración: ")
print(cadena)
print("Oración convertida a lista: ")
print(listaCadena)

# El comando .split te permite convertir las palabras de una cadena en una lista
# SIEMPRE Y CUANDO estén separadas por espacios.
```

Resultado:

```
Oración:
me gustan las albondigas
Oración convertida a lista:
['me', 'gustan', 'las', 'albondigas']
```

## Comentarios con una línea

```
# Este comando te permite borrar algo eligiendo la posición.
```

## Comentarios multilínea

```
"""
Como se puede ver aquí, hemos hecho una DeepCopy de la lista 1 y es totalmente
independiente, es decir que por muchos cambios que le haga ahora a la lista 1 no se
replicarán en la lista 3
"""
```

## Actividad 02

### Función que reciba dos números y devuelva la suma

```
# Función que recibe dos números y los suma

def suma(a, b):
    return a + b

numSuma1 = int(input("Dame un número para sumar: "))
numSuma2 = int(input("Dame otro número para sumar: "))

resultado = suma(numSuma1, numSuma2)
print("La suma ha dado: ")
print(resultado)

# Para definir una función ponemos 'def', el nombre de la función y las variables
# que recibe. Tras eso dentro de la función puedes hacer lo que quieras y con return
# devolver lo que quieras, tampoco tienes porqué devolver nada.
```

También añadir: input hace que la consola reciba un dato para usarlo y lo que pongas entre paréntesis es la pregunta que le haces al usuario.

Resultado:

```
Dame un número para sumar: 6
Dame otro número para sumar: 7
La suma ha dado:
13
```

### Función que recibe una lista y la modifique doblando sus valores sin devolver nada

```
listaFuncion = [10, 20, 30, 40, 50]

def doblar_lista(listaFuncion):
    for i in range(len(listaFuncion)):
        listaFuncion[i] = listaFuncion[i] * 2

print("Lista antes de doblar")
print(listaFuncion)

doblar_lista(listaFuncion)

print("Lista doblada")
print(listaFuncion)

# La función coge una lista en el () y con ella hace lo que quiera, en este caso
# usa el for each para que cada elemento sea multiplicado por 2
```

No tiene return, ya que esta función modifica permanentemente la lista.

Resultado:

```
Lista antes de doblar
[10, 20, 30, 40, 50]
Lista doblada
[20, 40, 60, 80, 100]
```

Función que recibe una lista, la copia y la dobla sin modificar la original

```
listaFuncionCopia = [0]

def copia_doblant(lista):
    copia = lista.copy()

    for i in range(len(copia)):
        copia[i] = copia[i] * 2

    return copia

listaFuncionCopia = copia_doblant(listaFuncion)

print("Lista original")
print(listaFuncion)
print("Lista copiada y doblada")
print(listaFuncionCopia)

# Esta función coge una lista, la copia y luego hace lo mismo que la anterior, esta
# vez te devuelve (return) la copia para que la guardes, en vez de modificar la
# original permanentemente
```

Resultado:

```
Lista original
[20, 40, 60, 80, 100]
Lista copiada y doblada
[40, 80, 120, 160, 200]
```

## Actividad 03

Almacenar un usuario con su contraseña pasándolo a Hash

Usando una lista

```
# Vamos a guardar usuarios y contraseñas utilizando una lista.
# Las contraseñas se guardarán en formato Hash mediante SHA-256.

# Función que convierte una contraseña a Hash
def hacer_hash(contraseña):
    return hashlib.sha256(contraseña.encode()).hexdigest()

# Lista con 5 usuarios y sus contraseñas en Hash
usuarioslista = [
    ["juan", hacer_hash("1234")],
    ["maria", hacer_hash("abcd")],
    ["pedro", hacer_hash("python")],
    ["laura", hacer_hash("clave123")],
    ["ana", hacer_hash("hola")]
]

print("Lista de usuarios y contraseñas:")
print(usuarioslista)
```

```
# Función para consultar un usuario en la lista
def consultar_lista(usuario, contraseña):
    contraseñaHash = hacer_hash(contraseña)

    for usuariolista in usuarioslista:
        if usuariolista[0] == usuario and usuariolista[1] == contraseñaHash:
            return True

    return False

# Hacemos dos consultas
print("Consulta 1:")
print(consultar_lista("juan", "1234"))

print("Consulta 2:")
print(consultar_lista("juan", "contraseñaIncorrecta"))
```

Hashlib es un módulo de Python para funciones Hash, sha256() toma los bytes y calcula un hash SHA-256.

Al hacer una consulta, si la contraseña y el usuario coinciden da True, de otra forma, False.

Resultado:

```
Lista de usuarios y contraseñas:
[['juan', '03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4'], ['maria', '88d4266fd4e6338d13b845fcf289579d209c897823b9217da3e161936f031589'], ['pedro', '11a4a60b518bf24989d481468076e5d5982884626aed9faeb35b8576fcd223e1'], ['laura', '5ac0852e770506dcd80f1a36d20ba7878bf82244b836d9324593bd14bc56dcb5'], ['ana', 'b221d9ddb083a7f33428d7c2a3c3198ae925614d70210e28716ccaa7cd4ddb79']]
Consulta 1:
True
Consulta 2:
False
```

(la consola es más extensa pero simplemente son más contraseñas encriptadas)

## Usando un diccionario

```
# Ahora hacemos lo mismo utilizando un diccionario.
# En este caso el usuario será la clave y el Hash de la contraseña será el valor.

usuariosDiccionario = {
    "juan": hacer_hash("1234"),
    "maria": hacer_hash("abcd"),
    "pedro": hacer_hash("python"),
    "laura": hacer_hash("clave123"),
    "ana": hacer_hash("hola")
}

print("Diccionario de usuarios y contraseñas:")
print(usuariosDiccionario)

# Función para consultar un usuario en el diccionario
def consultar_diccionario(usuario, contraseña):
    contraseñaHash = hacer_hash(contraseña)

    if usuario in usuariosDiccionario:
        if usuariosDiccionario[usuario] == contraseñaHash:
            return True

    return False
```

```
# Hacemos dos consultas
print("Consulta 1:")
print(consultar_diccionario("maria", "abcd"))

print("Consulta 2:")
print(consultar_diccionario("maria", "contraseñaIncorrecta"))
```

Resultado:

```
Diccionario de usuarios y contraseñas:
{'juan': '03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4', 'maria': '88d4266fd4e6338d13b845fcf289579d209c897823b9217da3e161936f031589', 'pedro': '11a4a60b518bf24989d481468076e5d5982884626aed9faeb35b8576fcd223e1', 'laura': '5ac0852e770506dcd80f1a36d20ba7878bf82244b836d9324593bd14bc56dcb5', 'ana': 'b221d9dbb083af33428d7c2a3c3198ae925614d70210e28716ccaa7cd4ddb79'}
Consulta 1:
True
Consulta 2:
False
```

## Actividad 04

### Operador “is”

```
# OPERADOR "is"
# El operador "is" comprueba si dos variables hacen referencia al mismo objeto.
# No comprueba simplemente si tienen el mismo contenido.

listaIs1 = [1, 2, 3]
listaIs2 = listaIs1
listaIs3 = [1, 2, 3]

print("¿listaIs1 y listaIs2 son el mismo objeto?")
print(listaIs1 is listaIs2)

print("¿listaIs1 y listaIs3 son el mismo objeto?")
print(listaIs1 is listaIs3)

# Como listaIs2 apunta a la misma lista que listaIs1, devuelve True.
# Aunque listaIs3 tenga exactamente los mismos elementos, es otra lista
# diferente, por lo que devuelve False.
```

Resultado:

```
¿listaIs1 y listaIs2 son el mismo objeto?
True
¿listaIs1 y listaIs3 son el mismo objeto?
False
```

## Operador “not”

```
# OPERADOR "not"
# El operador "not" sirve para negar una condición.
# Si algo es True, "not" lo convierte en False.
# Si algo es False, "not" lo convierte en True.

numero = 10

print("¿El número NO es igual a 10?")
print(not numero == 10)

print("¿El número NO es igual a 5?")
print(not numero == 5)
```

Resultado:

```
¿El número NO es igual a 10?
False
¿El número NO es igual a 5?
True
```

## Operador “in”

```
# OPERADOR "in"
# El operador "in" sirve para comprobar si un elemento está dentro
# de una lista, cadena de texto, diccionario, etc.

frutas = ["manzana", "pera", "plátano"]

print("¿La manzana está en la lista?")
print("manzana" in frutas)

print("¿La naranja está en la lista?")
print("naranja" in frutas)

# También podemos utilizar "not in" para comprobar que un elemento
# NO está dentro de una lista.

print("¿La naranja NO está en la lista?")
print("naranja" not in frutas)
```

Resultado:

```
¿La manzana está en la lista?  
True  
¿La naranja está en la lista?  
False  
¿La naranja NO está en la lista?  
True
```

## Actividad 05

### Introducir parámetros

```
# PARÁMETROS DESDE LA CONSOLA  
  
import sys  
  
# Con sys.argv podemos recoger los parámetros que ponemos  
# al ejecutar el programa desde la consola.  
  
# Por ejemplo:  
# python Actividad01UD00AngelEstebanAct5.py Juan 20 Valencia  
  
# sys.argv[0] será el nombre del programa.  
# sys.argv[1] será Juan.  
# sys.argv[2] será 20.  
# sys.argv[3] será Valencia.  
  
print("Nombre del programa:")  
print(sys.argv[0])  
  
print("Parámetros introducidos:")  
print(sys.argv[1:])
```

```
# Podemos acceder a cada parámetro utilizando su posición.

if len(sys.argv) > 1:
    print("Primer parámetro:")
    print(sys.argv[1])

if len(sys.argv) > 2:
    print("Segundo parámetro:")
    print(sys.argv[2])

if len(sys.argv) > 3:
    print("Tercer parámetro:")
    print(sys.argv[3])
```

Resultado:

```
Nombre del programa:
Actividad01UD00AngelEstebanAct5.py
Parámetros introducidos:
['Juan', '20', 'Valencia']
Primer parámetro:
Juan
Segundo parámetro:
20
Tercer parámetro:
Valencia
```

## Sobrecarga de funciones

```
# SOBRECARGA DE FUNCIONES

# En Python podemos hacer que una función reciba un número
# indefinido de parámetros utilizando *.

def mostrar_parametros(*parametros):
    print("Parámetros recibidos:")

    for parametro in parametros:
        print(parametro)

# Podemos pasar diferentes cantidades de parámetros.

mostrar_parametros("Hola")

mostrar_parametros("Hola", "Adiós")

mostrar_parametros("Juan", 20, "Valencia")

# También podemos no pasar ningún parámetro.

mostrar_parametros()
```

Resultado:

```
Parámetros recibidos:
Hola
Parámetros recibidos:
Hola
Adiós
Parámetros recibidos:
Juan
20
Valencia
Parámetros recibidos:
(SGE) PS C:\Users\angel\Downloads\SGE> █
```

## Actividad 06

```
# Creamos una lista en la que cada elemento tiene dos valores:
# el primero será la altura y el segundo será el peso.

personas = [
    [175, 70],
    [180, 80],
    [165, 60],
    [180, 75],
    [170, 65],
    [175, 68]
]

print("Lista original:")
print(personas)

# Para ordenar la lista utilizamos sorted().
# Queremos que la altura vaya de mayor a menor y que,
# en caso de tener la misma altura, el peso vaya de menor a mayor.

def ordenar_persona(persona):
    return (-persona[0], persona[1])

personasOrdenadas = sorted(personas, key=ordenar_persona)
```

```
print("Lista ordenada:")
print(personasOrdenadas)

# Una key function es una función que recibe un elemento de la lista
# y devuelve el valor que queremos utilizar para ordenar ese elemento.

# En este caso recibe una persona, que es una lista con altura y peso.
# Devuelve (-altura, peso).

# Ponemos la altura en negativo para conseguir que se ordene
# de mayor a menor.

# El peso lo dejamos positivo para que, en caso de empate de altura,
# se ordene de menor a mayor.

# La función key se ejecuta una vez para cada elemento de la lista
# y Python utiliza los valores que devuelve para realizar la ordenación.
```

Resultado:

```
Lista original:
[[175, 70], [180, 80], [165, 60], [180, 75], [170, 65], [175, 68]]
Lista ordenada:
[[180, 75], [180, 80], [175, 68], [175, 70], [170, 65], [165, 60]]
```

## Actividad 07

```
import random

# Creamos la clase Car.
# Cada coche tendrá una matrícula y un color.

class Car:

    # Este método se utiliza para crear cada coche.
    def __init__(self, matricula, color):
        self.matricula = matricula
        self.color = color

    # Método para imprimir los datos del coche.
    def imprimir(self):
        print("Matrícula:", self.matricula)
        print("Color:", self.color)

    # Primer método que hemos añadido.
    def arrancar(self):
        print("El coche con matrícula", self.matricula, "ha arrancado.")

    # Segundo método que hemos añadido.
    def parar(self):
        print("El coche con matrícula", self.matricula, "se ha parado.")
```

```

# Pedimos al usuario cuántos coches quiere crear.

n = int(input("¿Cuántos coches quieres crear? "))

# Creamos la lista de colores que podremos utilizar.

colores = ["red", "white", "black", "pink", "blue"]

# Creamos una lista donde guardaremos todos los coches.

coches = []

# Creamos tantos coches como haya indicado el usuario.
# La matrícula será un número consecutivo empezando desde 1.
# El color se elegirá aleatoriamente de la lista de colores.

for i in range(1, n + 1):
    color = random.choice(colores)

    coche = Car(i, color)

    coches.append(coche)

```

```

# Finalmente imprimimos las primeras 10 instancias.
# Si n es menor que 10, solamente imprimiremos las que existan.

cantidad = min(n, 10)

for i in range(cantidad):
    print("Coche", i + 1)
    coches[i].imprimir()
    print()

for coche in coches:
    coche.arrancar()
    coche.parar()

```

Resultado:

```
¿Cuántos coches quieres crear? 5  
Coche 1  
Matrícula: 1  
Color: black
```

```
Coche 2  
Matrícula: 2  
Color: white
```

```
Coche 3  
Matrícula: 3
```

```
Coche 4  
Matrícula: 4  
Color: pink
```

```
Coche 5  
Matrícula: 5  
Color: red
```

```
El coche con matrícula 1 ha arrancado.  
El coche con matrícula 1 se ha parado.  
El coche con matrícula 2 ha arrancado.  
El coche con matrícula 2 se ha parado.
```

```
El coche con matrícula 1 ha arrancado.  
El coche con matrícula 1 se ha parado.  
El coche con matrícula 2 ha arrancado.  
El coche con matrícula 2 se ha parado.  
El coche con matrícula 3 ha arrancado.  
El coche con matrícula 3 se ha parado.  
El coche con matrícula 4 ha arrancado.  
El coche con matrícula 4 se ha parado.  
El coche con matrícula 5 ha arrancado.  
El coche con matrícula 5 se ha parado.
```

[Enlace GitHub](#)